

Fase 2 — Notificar e a simplificação dos workflows · status de entrega

Sem número de bloco. Trilha da automação, lista "OCR Contratos (WhatsApp) — Breno" do ClickUp, numeração própria de 1 a 8. Ver docs/README.md e o CLAUDE.md.

Data: 2026-09-22 · Branch: feature-dev-breno-automacao · Ambiente: n8n Cloud ederbox.app.n8n.cloud e gc-dev (gzbmhgnpoehormnidmgg) · Plano: docs/automacao/migracao-telegram-integracao-automacao.md , Fase 2 e decisões 14 a 19.

Resumo

Fase 2 entregue e aceita, e a simplificação que substitui a Fase 3 está concluída e testada de ponta a ponta com PDF real. O n8n passou de 8 workflows para 3: Notificar , Processar Documento e o SLA (inativo). Nenhum deles cita Z-API e nenhum tem segredo escrito nos nós.

O fluxo que a rodada entrega agora é o que o Breno definiu em 22/09: **Telegram recebe → triagem → a proposta sobe no sistema e aparece em /propostas** . Só propostas; contrato fica para outra rodada. O pedaço "Telegram recebe" é a Fase 4, e entra como gatilho dentro do próprio Processar Documento — o alvo final é 2 workflows.

O escopo mudou três vezes durante a rodada, sempre por decisão do Breno, registradas no plano:

DECISÃO	O QUÊ
14	Z-API sai por completo, sem rollback
15	Poucos workflows, o mais simples possível
16	Proposta e contrato se ligam à obra
17	A Resolução Manual de Revisão sai
18	Nesta rodada o canal recebe só propostas
19	Na Fase 4 o Telegram vira gatilho do Processar Documento

1. Estado final do n8n

WORKFLOW	ID	ESTADO	NÓS
Obraminds - Notificar	zblwWrHmis3GNG KU	ativo	8

WORKFLOW	ID	ESTADO	NÓS
Obraminds - Processar Documento	th1KSZcBPX84Y5 1N	ativo (sem gatilho externo até a Fase 4)	14
Obraminds - SLA Gatilhos de Execução (5 dias)	TksH6ervKPA7Li ar	inativo (decisão 8)	7

Apagados em 22/09 — 10 workflows:

TIPO	WORKFLOWS
Produção (7)	Ingestão via WhatsApp (Z-API), Ingestão e Extração, Triagem, Integração Oficial (Contrato), Integração Oficial (Proposta), Resolução Manual de Revisão, Notificação de Erro
Descartáveis de teste (3)	o da Fase 0 e dois chamadores temporários desta fase

Backup dos 8 workflows de produção, feito antes de qualquer remoção, em `/Users/a1234/n8n-backup/2026-09-22/` — fora do repositório, porque os nós antigos carregavam a chave secreta do Supabase e a chave do Gemini em texto claro.

Segredos: varredura dos 3 workflows restantes procurando chave do Supabase, chave do Gemini e JWT: **0**. Os nós usam as Credentials criadas pelo Breno — Supabase — `gc-dev (service)` (`t6DCsidxPWw7kdQX`) e Gemini — extração de documentos (`lPCf0dmksKJC1aTq`) — além das duas do Telegram da Fase 0. O SLA também foi trocado para a Credential: era o último lugar com a chave do Supabase escrita no nó. Continua inativo; o `errorWorkflow` dele, que apontava para a Notificação de Erro apagada, agora aponta para o `Notificar`.

Referências a Z-API/WhatsApp: 0 chamadas. As únicas ocorrências da palavra estão em dois comentários de código do `Notificar` explicando o que ele substituiu.

2. Fase 2 — Obraminds - Notificar

Execute Sub-workflow com `inputSource: passthrough`, contrato de entrada do plano:

```
{ destino: 'CLIENTE'|'ADMIN', canal: 'TELEGRAM', chat_id, telefone, texto,
  opcoes: null, contexto: { documentoId, empresaId, origem } }
```

- **Só Telegram** (decisão 14). Qualquer outro `canal` é descartado com motivo, sem enviar.
- O `chat_id` do grupo de admin mora em um único nó, `Resolver destino` (decisão 5).
- **Envia pelo bot de produção**, porque é com ele que os clientes vão falar, e o Telegram só deixa um bot escrever no privado de quem deu `/start` nele.

- **Texto puro**, sem Markdown e sem a assinatura "sent automatically with n8n".
- **Falha de envio não derruba quem chamou** (`onError: continueRegularOutput`): devolve `{ entregue: false, erro }`.
- **Absorveu a Notificação de Erro**: um Error Trigger manda `[ERRO DE SISTEMA]` ao grupo quando um workflow que aponta para o `Notificar` falha sem tratamento.

Aceite

CHAMADA	RESULTADO
ADMIN → grupo	entregue (<code>message_id 6</code>), conferido na tela do grupo pelo Breno
CLIENTE → privado, antes do <code>/start</code> no bot de produção	<code>Forbidden: bot can't initiate conversation with a user</code> , e a execução terminou <code>success</code> — a recusa não quebrou nada
canal <code>WHATSAPP</code>	descartada sem enviar
CLIENTE → privado, depois do <code>/start</code>	entregue (<code>message_id 8</code>)
<code>n8n_validate_workflow</code> : 0 erros, 0 warnings.	

3. Obraminds - Processar Documento

Um workflow, **14 nós**, no lugar de 4 que somavam **72** (Ingestão e Extração 23, Triagem 23, Integração Contrato 15, Integração Proposta 11).

```

Trigger → Normalizar entrada → Criar registro (PENDENTE) → Baixar PDF → base64
  → Gemini (5 tentativas) → Analisar e decidir → Proposta aprovada?
    sim → Criar proposta → Documento aprovado —————|
    não → Documento em revisão —————|
qualquer falha → Registrar falha → Documento em revisão |
                                     Montar avisos → Notificar

```

- **As 9 bifurcações da antiga Triagem viraram um Code node** (`Analisar e decidir`): a regra inteira num lugar legível.
- **Os avisos Z-API viraram um par**: `Montar avisos` (todos os textos, cliente e admin) → `Notificar`.
- **Entrada**: `{ empresa_id, obra_id, arquivo_url, canal, chat_id, numero_contrato? }`. Na Fase 4, o Telegram Trigger entra na frente do `Normalizar entrada`.

A regra, como está

SITUAÇÃO	DESFECHO	O QUE O CLIENTE RECEBE
Proposta com valor total, obra e número	APROVADO , proposta criada	"Sua proposta nº X foi registrada no sistema." (+ pedido dos campos que faltarem)
Documento classificado como contrato	REVISAO_HUMANA , nada criado	"...por enquanto recebemos por aqui somente propostas..."
Proposta sem valor total	ERRO_VALIDACAO	pedido do valor
Sem obra	REVISAO_HUMANA	"...não conseguimos confirmar a que obra..."
Sem número da proposta	REVISAO_HUMANA	pedido do número
Número já existente na empresa	REVISAO_HUMANA	"já existe uma proposta com o número X..." (decisão 9)
Falha técnica em qualquer etapa	REVISAO_HUMANA com o erro	"Tivemos um problema..."

Em todos: o grupo de admin recebe o aviso correspondente, e **nenhum documento fica preso em PENDENTE** .

A proposta é criada com: o número do próprio documento (decisão 9), a **obra** do contato (decisão 16), `status: enviada` , `valor_total` , condições de pagamento, cliente e entrega na observação, e `created_by` = profile de serviço "Automação (Telegram)" (decisão 12). O documento guarda `proposta_criada_id` , `canal` e `canal_chat_id` .

O prompt do Gemini foi preservado **palavra por palavra** — a Fase 5 é quem vai mexer nele.

4. Testes de ponta a ponta

Chamador temporário por webhook (apagado depois), com PDFs reais do storage do gc-dev. Empresa LC EMPRESA, obra `TESTE OBRA 3` , `chat_id` do Breno — os avisos de cliente chegaram no privado dele, os de admin no grupo.

4.1 Primeira bateria — 7 execuções, com o ramo de contrato ainda presente

#	CAMINHO	RESULTADO
1	falha no download	REVISAO_HUMANA — URL de teste mal assinada (ver 4.3)
2-3	falha no Gemini	REVISAO_HUMANA — Gemini 503 "model experiencing high demand"
4	proposta aprovada	APROVADO , proposta criada, <code>proposta_criada_id</code> preenchido
5	contrato duplicado	REVISAO_HUMANA com motivo claro

#	CAMINHO	RESULTADO
6	proposta duplicada	REVISAO_HUMANA com motivo claro (decisão 9)
7	contrato sem número	REVISAO_HUMANA com motivo claro

4.2 Reteste depois da decisão 18 (só propostas) — 2 execuções

#	PDF	RESULTADO
8	proposta EB-26-04-0157 (AL+VD)	APROVADO . Proposta criada: R\$ 797.263,62, obra TESTE OBRA 3 , status: enviada , autor "Automação (Telegram)"
9	contrato P O N - R\$ 156.000,00	REVISAO_HUMANA , motivo "por enquanto recebemos por aqui somente propostas...", nenhum contrato criado

"Mostra" — **provado no nível do dado, não da tela**. Logado como `breno@obraminds.com` , sob RLS, o `select` copiado de `src/app/(app)/propostas/page.tsx:64-67` (com o join de obra e cliente) devolveu a proposta — 1 linha, obra e valor final certos — e a leitura do detalhe `/propostas/[id]` também. O `next dev` **não** foi subido para um screenshot, de propósito: ele regrava o `.next` , e a sessão da sprint 5 trabalha na mesma pasta.

Limpeza, nas duas baterias: documentos e propostas de teste apagados por id. gc-dev ficou em **29 documentos, 0 com canal Telegram, 0 propostas da automação** — o estado de antes.

4.3 Defeitos que os testes acharam, corrigidos

1. `$(variavel)` em Code node chega vazio. O n8n 2.x só manda ao Code node os dados dos nós citados **literalmente**. Trocado por referências literais.
2. `$('#Criar registro').first()` **lia o ramo errado** — o de erro, vazio, que também leva ao `Montar avisos` . Corrigido com `first(0)` . O aviso ao admin saía "Documento null".
3. **Gemini sem retry**. Agora **5 tentativas, 5 s entre elas** (o máximo do n8n).
4. **Motivo técnico genérico** ("Service unavailable"). Agora grava também a descrição real.
5. **Contrato duplicado caía como falha técnica** (irrelevante desde a decisão 18, mas corrigido antes dela).
6. Duas concordâncias nas mensagens.

5. Números reais

VERIFICAÇÃO	RESULTADO
Workflows no n8n	8 → 3
Workflows apagados	10 (7 de produção, 3 descartáveis)

VERIFICAÇÃO	RESULTADO
Workflows com backup JSON antes da remoção	8
Nós do pipeline	72 → 14
Segredos escritos nos nós dos 3 restantes	0
Validação <code>Notificar</code> / <code>Processar Documento</code>	0 erros, 0 warnings nos dois
Chamadas de teste ao <code>Notificar</code>	4 — 2 entregues, 1 recusada sem quebrar, 1 descartada
Execuções do <code>Processar Documento</code> com PDF real	9
Avisos entregues nessas 9	18 (cliente + admin em cada uma)
Colunas gravadas conferidas em gc-dev	45/45 existem (antes de tirar o ramo de contrato)
Proposta visível sob RLS pela consulta da <code>/propostas</code>	1/1
Linhas de teste em gc-dev depois da limpeza	0

6. O que NÃO foi validado — pendências nominais

1. A tela `/propostas` não foi aberta. O dado aparece para o usuário logado pela mesma consulta da página (4.2); o render não foi visto. É um screenshot com `bash scripts/validar.sh navegador` ou a skill `run-gc-sistema`, quando a sessão da sprint 5 não estiver usando o `.next`.
2. Nada chama o `Processar Documento` ainda. O gatilho do Telegram é a Fase 4.
3. A proposta ainda é gravada por REST direto, não pela rota do sistema (decisão 1). É a Fase 6. Os itens da proposta não são gravados — o Gemini extrai, o fluxo não grava. É Fase 5 (extração) e 6 (gravação).
4. Os textos das mensagens foram redigidos de novo. O plano pedia textos 1:1 na Fase 3; com a reescrita, os de aprovação são novos (os de erro e revisão foram preservados). O Breno recebeu todos no celular durante o teste, mas ninguém os revisou como texto final.
5. O ramo de contrato foi removido, não arquivado em workflow. O desenho dele está no histórico de versões do n8n (`th1KSZcBPX84Y51N`) e na seção 4.1 deste documento.
6. O Error Trigger do `Notificar` nunca disparou.
7. `npm run validar` não rodou nesta rodada. Nenhum arquivo da aplicação foi tocado — só documentos. A última rodada completa foi na Fase 0, hoje (sete camadas, `exit 0`).
8. O grupo tem um membro "Deleted" — o bot antigo, apagado na Fase 0. Inofensivo.
9. `TELEGRAM_ADMIN_CHAT_ID` no `.env.local` era provisório "até a Fase 2 existir". Agora o número está no nó `Resolver destino`, e nada lê o do `.env.local`. Apagar ou manter é decisão do Breno.

Achados que vão para as próximas fases:

- **Fase 4:** o `+` no nome de um PDF (`AL+VD`) fez a URL assinada falhar com `InvalidSignature` . A ingestão deve **higienizar o nome do arquivo** ao gravar no storage.
- **Fase 5:** um PDF veio com item de valor zerado — o `valor_unit` ausente da decisão 10, visto em dado real.

7. Próximo passo

Fase 4: Telegram Trigger dentro do `Processar Documento` . O bot recebe o PDF, acha o contato pelo `telegram_chat_id` (empresa e obra), grava o arquivo no storage com nome higienizado, e segue pelo fluxo que já está testado. Contato desconhecido recebe o próprio `chat_id` para passar ao admin (decisão 4). Ao fim, o sistema tem **2 workflows**.